



Driver safety and Monitoring System

Ahmed Abohadeed , Ali Nabil , Idris tarek ,Ahmed Gouda , ALSayed sameh
Rania Yasser , Khloud Elsayed ,Shereen Ebrahim , Eman hamada
Supervised by : Dr / Mostafa Abdelsallam
Eng/ Sara Abdel Mohsen



Benha University

Abstract

Our main project is a Driver's Safety Monitoring System, which aims to improve the car's safety while the car is on the move. Improving the safety of the car during motion will be through monitoring the driver's behavior, which consists of the driver's yawning blinking and head nodding. Tracking the driver's behavior, the system's job is to detect the driver's drowsiness level and take the suitable action according to the driver's drowsiness level.

Objectives

This system's goal is to enhance the driver's safety mainly by detecting his/her drowsiness level while driving. Since the drowsiness level could be vary, according to that level, the system is going to take suitable measures to ensure the driver's and the surrounding's safety. These measures will be alerts through the screen in a form of messages, on which the driver can respond, and through the car's sound system in a form of high sounds.

Methods

Facial Landmark Detection

We Explore three techniques for detecting facial landmarks:
Dlib: Known for its precision but computationally intensive.
Haarcascade: Faster but less accurate.
Mediapipe: Outperformed the other two, providing the best balance of speed and accuracy.



Eye and Mouth Detection

We explored two approaches:
Custom-Built Models: Designed and trained from scratch for this specific task.
Transfer Learning Models: Pre-trained models adapted for our needs.
Our custom-built models delivered superior accuracy compared to the transfer learning models.

Classification Techniques

We employed two distinct methods for classification:
Model-Based Classification: Using our custom-built models to determine eye and mouth states.
Parametric Approach: A rule-based system to classify whether eyes are open or closed and detect yawning. This approach proved to be faster in detection.

Deployment Strategy

To ensure efficient and effective deployment, we undertook the following steps:
Model Conversion: Converted our models to TensorFlow Lite (tflite) format for optimized performance on embedded devices.
C++ Inference Engine: Developed a C++ inference engine to execute the tflite models, significantly enhancing processing speed.
Mediapipe Integration: Integrated Mediapipe tflite models directly without using the framework, streamlining the process.
Human-Machine Interface (HMI): Built using QT C++, providing drivers with real-time results and interaction capabilities.

Communication and interface :

- Control GSM/SIM808 with AT commands for calls and location messages
- RTSP (Real-Time Streaming Protocol) for real-time video streaming
- MQTT (Message Queuing Telemetry Transport) for efficient message exchange between devices

Hardware Component

A System-on-a-Chip (SoC) board: is the core of the hardware components since every hardware component is connected to it .used for connecting to other components in addition to several other pins and ports such as the HDMI port for display. It supports high-resolution sensors and can process several sensors in parallel. Ex :- Raspberry pi Model B

Screen: To communicate with the driver behind the wheels

SIM 808 GPS/GSM Module: Since the driver could be already not responding to the alert messages on the screen, a fast action is needed to be taken. To minimize the damage and to prevent the loss of innocent lives, an SOS signal is sent to the emergency operator through the modem. The driver can make an emergency call.

Camera: the camera used for detecting the frames of the driver and send it to the board to make the processing we use **Raspberry pi HQ Camera and webcam**

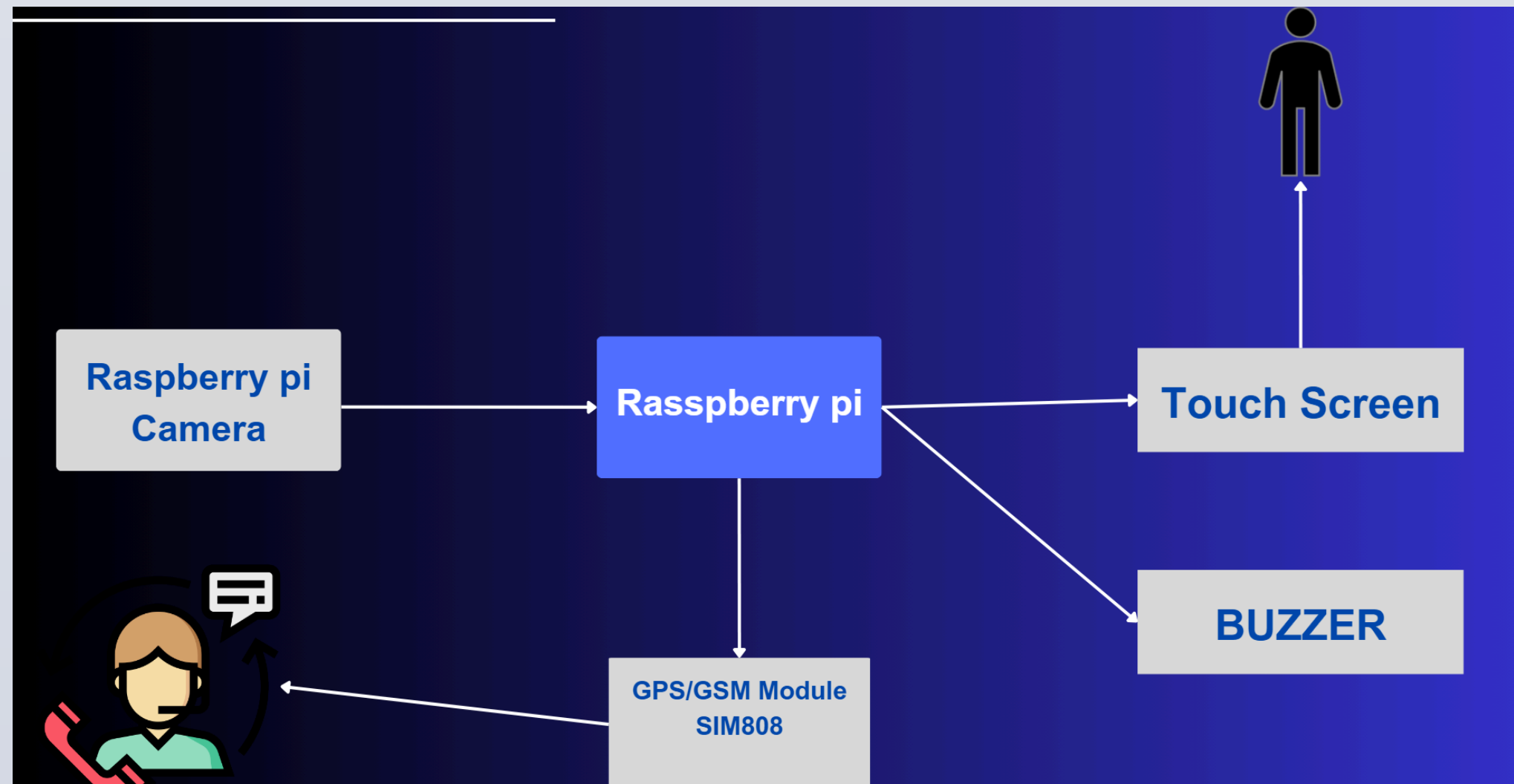
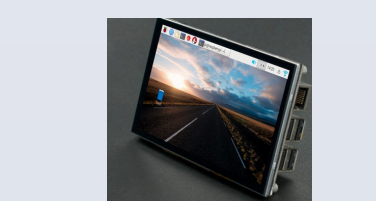
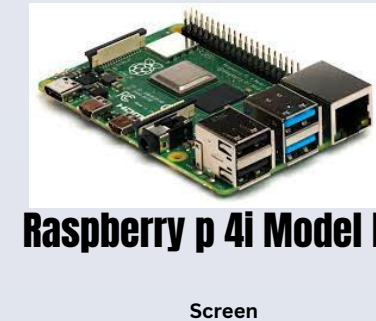


Figure 1: Hardware level 0

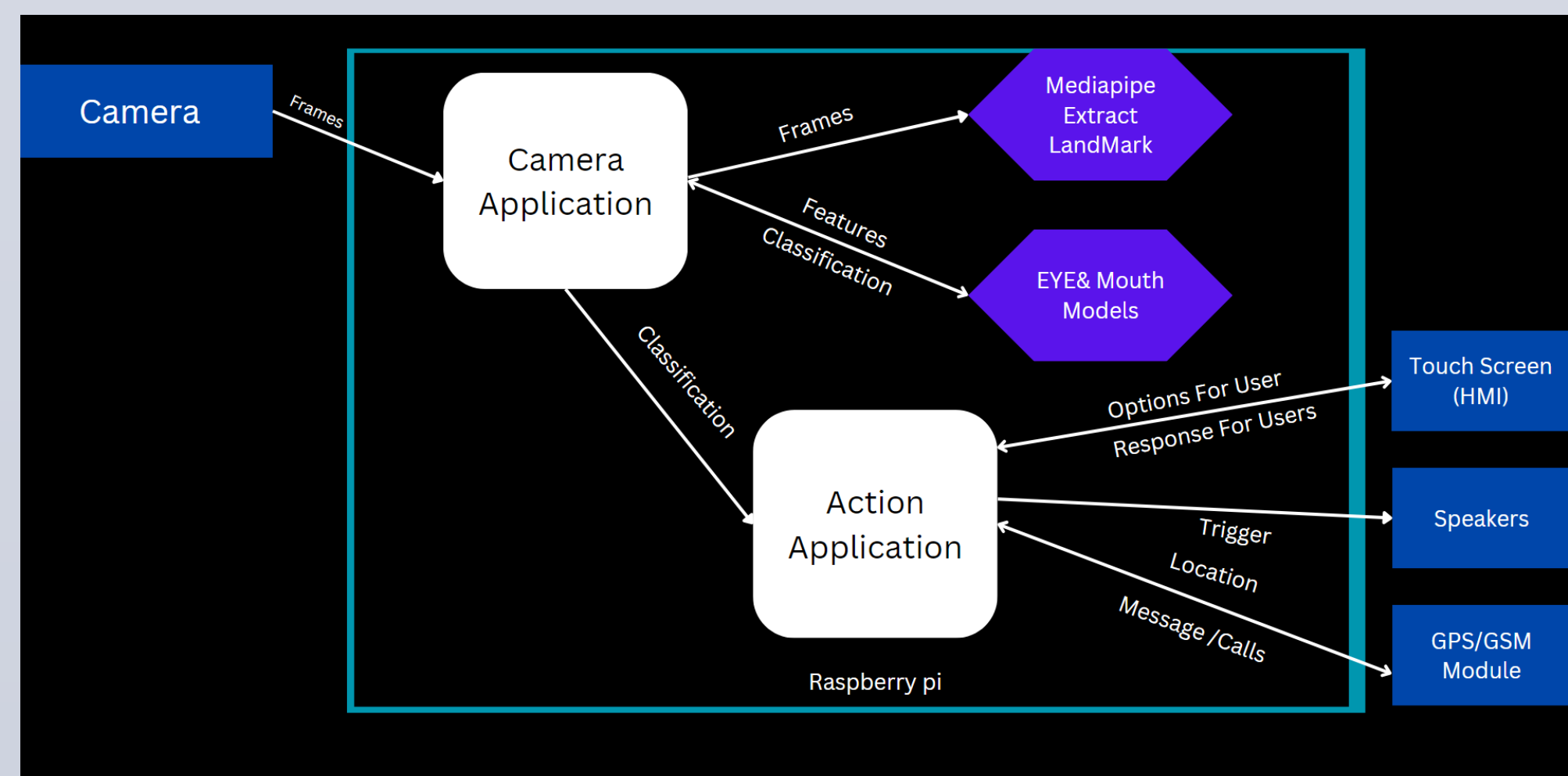


Figure2

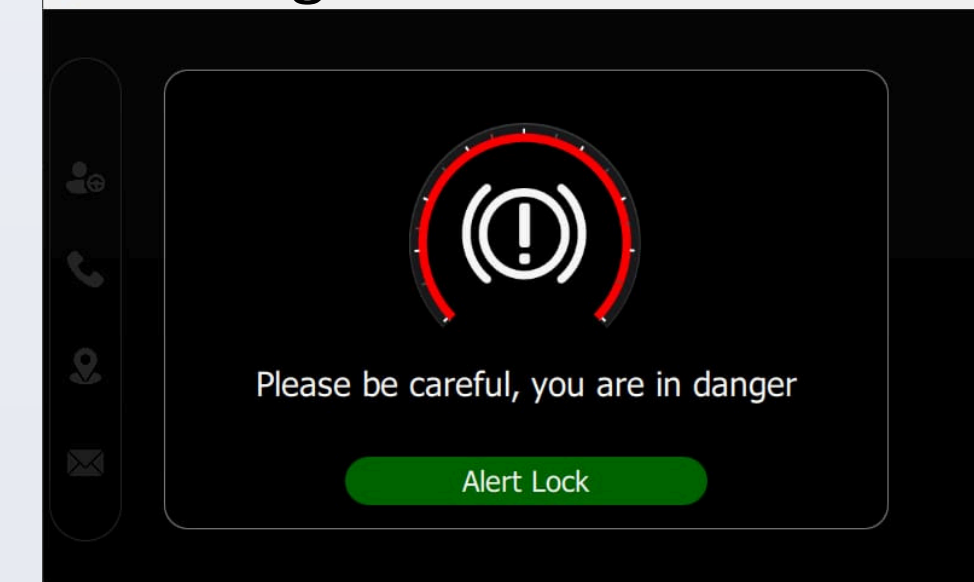
Result

HMI(QT)

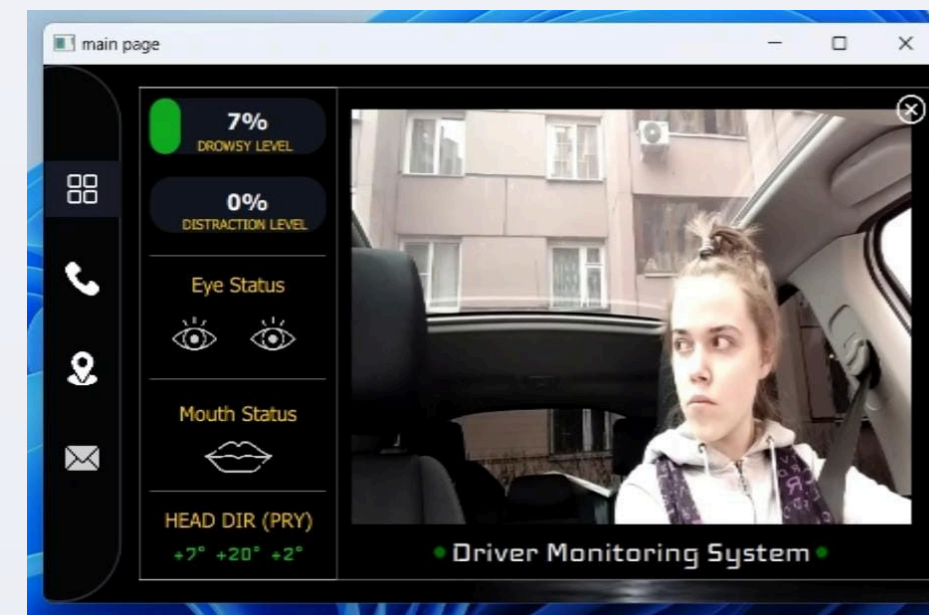
Main page : that appears in the start and level0(Normal)



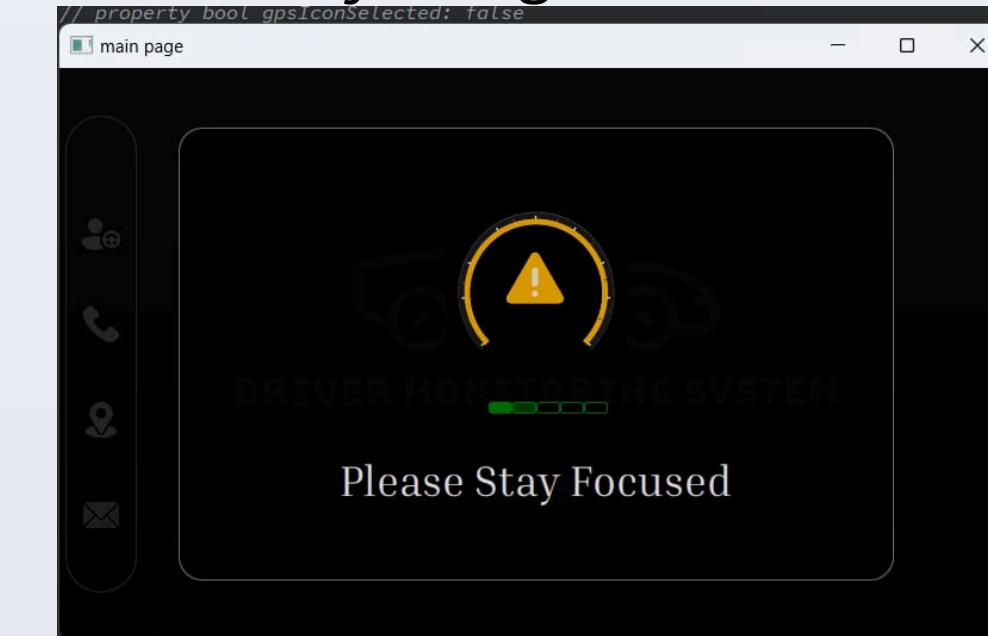
HIGH ALERT: appears When the Driver sleeps With high buzzing for 10 seconds



Debug page : to see models result

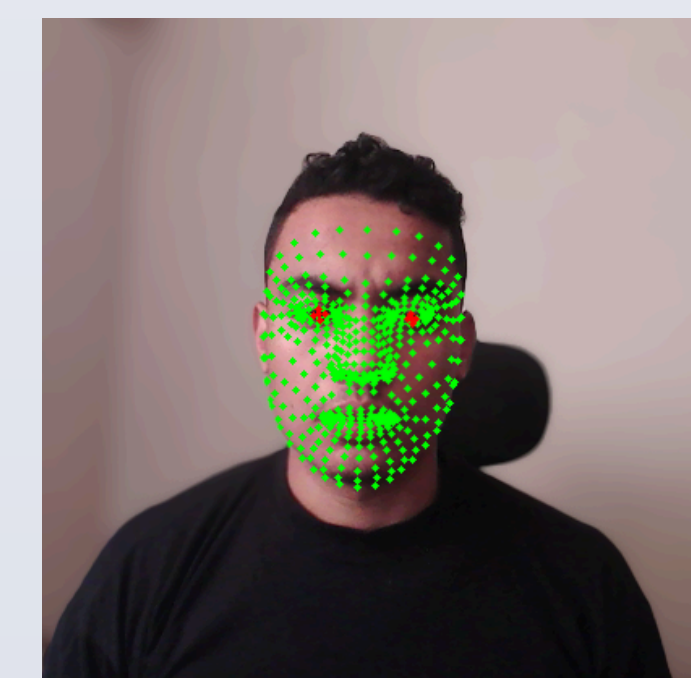


LOW ALERT: Appears when the driver begins to feel tired and start yawning

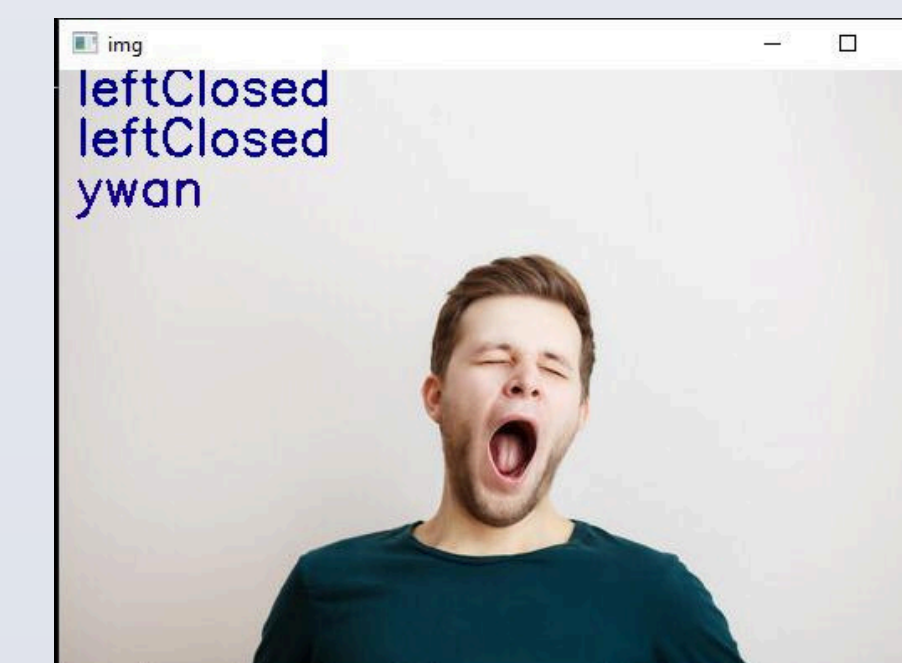


Mediapipe and models

mediapipe using C++ Facial land mark and Iris detection



The output for **models** and **parametric** inference



Conclusion

Our Driver Monitoring System leverages cutting-edge techniques in facial landmark detection and classification to ensure high accuracy and rapid response times. By integrating custom-built models with transfer learning and utilizing efficient deployment strategies, we have created a robust solution that outperforms existing methods. The combination of Mediapipe for landmark detection, TensorFlow Lite for model optimization, and a QT C++ interface for real-time driver interaction offers a comprehensive approach to driver safety. This system not only enhances the detection and classification of eye and mouth states but also ensures seamless communication and deployment on embedded devices, marking a significant advancement in driver monitoring technology.

Acknowledgements

We Would Like To Thank **Valeo** company for providing us with the necessary hardware for the project and the best guidance to complete the mission

